

N8N — Sender Emails Integration Guide

Overview

The Cold Email Pro platform supports **multiple sender emails per user**, each associated with an **email provider** (Resend, Zapmail, Amazon SES, Mailgun, custom SMTP, or manual). When a schedule is triggered, the webhook payload includes the full sender email config so N8N can route the dispatch through the correct provider API.

1. Database — Table: `sender_emails`

Run the migration `20260412120000_create_sender_emails_table.sql` in Supabase.

Full schema

```
CREATE TABLE sender_emails (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,  
  
  -- Identity  
  email_address text NOT NULL,  
  display_name text NOT NULL DEFAULT '',  
  domain text NOT NULL DEFAULT '',          -- extracted domain (company.com)  
  is_default boolean NOT NULL DEFAULT false,  
  
  -- Provider integration  
  provider text NOT NULL DEFAULT 'manual',   -- 'manual' | 'resend' | 'zapmail' | 'ses' |  
  'mailgun' | 'smtp'  
  provider_id text DEFAULT NULL,             -- ID of this email/domain in the external provider  
  status text NOT NULL DEFAULT 'active',     -- 'pending' | 'active' | 'error' | 'suspended'  
  provider_metadata jsonb NOT NULL DEFAULT '{}',  
  last_synced_at timestampz DEFAULT NULL,  
  
  created_at timestampz DEFAULT now(),  
  updated_at timestampz DEFAULT now()  
);  
  
-- Constraints & indexes  
ALTER TABLE sender_emails  
  ADD CONSTRAINT sender_emails_user_email_unique UNIQUE (user_id, email_address);  
  
CREATE UNIQUE INDEX sender_emails_one_default_per_user  
  ON sender_emails (user_id) WHERE is_default = true;  
  
CREATE INDEX sender_emails_provider_idx  
  ON sender_emails (user_id, provider);  
  
CREATE INDEX sender_emails_domain_idx  
  ON sender_emails (user_id, domain);
```

The `schedules` table also has a new FK column:

```
ALTER TABLE schedules  
  ADD COLUMN sender_email_id uuid REFERENCES sender_emails(id) ON DELETE SET NULL;
```

Column reference

Column	Type	Purpose
email_address	text	The actual from address (john@company.com)
display_name	text	"From" header name (John from Acme)
domain	text	Extracted domain for grouping (company.com)
provider	text	Which service hosts this email
provider_id	text	External ID in the provider (domain ID, API key ref, etc.)
status	text	pending / active / error / suspended
provider_metadata	jsonb	Flexible field for any provider-specific data
last_synced_at	timestampz	Last time status was verified with the provider

2. Provider Values

Value	Description	How N8N should send
manual	No API — email managed externally or via default SMTP	Use default SMTP credentials
resend	Hosted on Resend	Call Resend API (POST /emails) with API key
zapmail	Hosted on Zapmail	Call Zapmail API
ses	Amazon SES	Call SES SendEmail API
mailgun	Mailgun	Call Mailgun POST /messages API
smtp	Custom SMTP server	Use SMTP credentials from provider_metadata

provider_metadata examples

Resend:

```
{
  "api_key_ref": "resend_key_1",
  "domain_id": "d_abc123"
}
```

Custom SMTP:

```
{
  "host": "smtp.company.com",
  "port": 587,
  "secure": true,
  "auth_user": "john@company.com"
}
```

SMTP passwords should NOT be stored here — use N8N credentials or environment variables.

Amazon SES:

```
{
  "region": "us-east-1",
  "configuration_set": "campaign-tracking"
}
```

3. Updated Webhook Payload

When a schedule is triggered, the payload now includes full provider info:

```
{
  "schedule": true,
  "date": "2026-04-15T14:00:00.000Z",
  "schedule_id": "uuid",
  "schedule_name": "Weekly Campaign",
  "schedule_type": "one_time",
  "scheduled_date": "2026-04-15",
  "scheduled_time": "14:00",
  "recurring_days": [],
  "sender_email": {
    "id": "uuid",
    "email_address": "john@company.com",
    "display_name": "John from Company",
    "domain": "company.com",
    "provider": "resend",
    "provider_id": "d_abc123"
  },
  "emails": [
    {
      "id": "uuid",
      "email": "prospect@example.com",
      "company": "Acme Inc.",
      "lead_name": "Jane Doe",
      "sender_email": "john@company.com",
      "..."
    }
  ]
}
```

Key fields

Field	Type	Description
sender_email	object null	Selected sender for this schedule. null if none selected.
sender_email.provider	string	Which API to use for sending (resend, ses, manual, etc.)
sender_email.provider_id	string null	External ID for this email/domain in the provider
sender_email.domain	string	The domain part of the email
emails[].sender_email	string null	Per-lead sender override (legacy field, still populated)

4. N8N Implementation — Provider Routing

4.1. Routing logic (Switch node)

After the webhook trigger, use a **Switch** node on `sender_email.provider` :

```
sender_email.provider === "resend"    → Resend API node
sender_email.provider === "ses"       → SES API node
sender_email.provider === "mailgun"   → Mailgun API node
sender_email.provider === "smtp"      → SMTP Send node
sender_email.provider === "manual"    → Default SMTP node
sender_email === null                 → Default SMTP node (fallback)
```

4.2. Resend example

```
// HTTP Request node
// Method: POST
// URL: https://api.resend.com/emails
// Headers: Authorization: Bearer {{ $env.RESEND_API_KEY }}

{
  "from": "{{ $json.sender_email.display_name }} <{{ $json.sender_email.email_address }}>",
  "to": "{{ $json.email }}",
  "subject": "{{ $json.subject }}",
  "html": "{{ $json.body }}"
}
```

4.3. Generic SMTP fallback

```
const senderEmail = $json.sender_email;

const fromAddress = senderEmail?.email_address
  || $json.sender_email // per-lead field
  || 'default@yourcompany.com';

const fromName = senderEmail?.display_name || 'Your Company';

return {
  from_address: fromAddress,
  from_name: fromName,
};
```

4.4. Reading provider_metadata in N8N

If you need SMTP config from `provider_metadata` , query it from Supabase in N8N:

```
SELECT provider_metadata FROM sender_emails WHERE id = :sender_email_id;
```

The `provider_metadata` is a JSONB column — parse it in a Function node to extract host, port, etc.

5. N8N CRUD Workflows

5.1. List Sender Emails

```
SELECT * FROM sender_emails
WHERE user_id = :user_id
ORDER BY is_default DESC, created_at DESC;
```

5.2. Create Sender Email

```
INSERT INTO sender_emails (
  user_id, email_address, display_name, domain,
  provider, provider_id, status, provider_metadata
)
VALUES (
  :user_id, :email_address, :display_name, :domain,
  :provider, :provider_id, 'active', :provider_metadata::jsonb
)
RETURNING *;
```

5.3. Update Sender Email

```
UPDATE sender_emails
SET email_address = :email_address,
    display_name = :display_name,
    domain = :domain,
    provider = :provider,
    provider_id = :provider_id,
    status = :status,
    provider_metadata = :provider_metadata::jsonb,
    updated_at = now()
WHERE id = :id AND user_id = :user_id;
```

5.4. Update Status (after sync with provider)

```
UPDATE sender_emails
SET status = :new_status,
    last_synced_at = now(),
    updated_at = now()
WHERE id = :id;
```

5.5. Sync with Provider API

Create an N8N workflow that periodically:

1. Fetches all sender emails with `provider != 'manual'`
2. For each, calls the provider API to check domain/email verification status
3. Updates `status` and `last_synced_at` accordingly

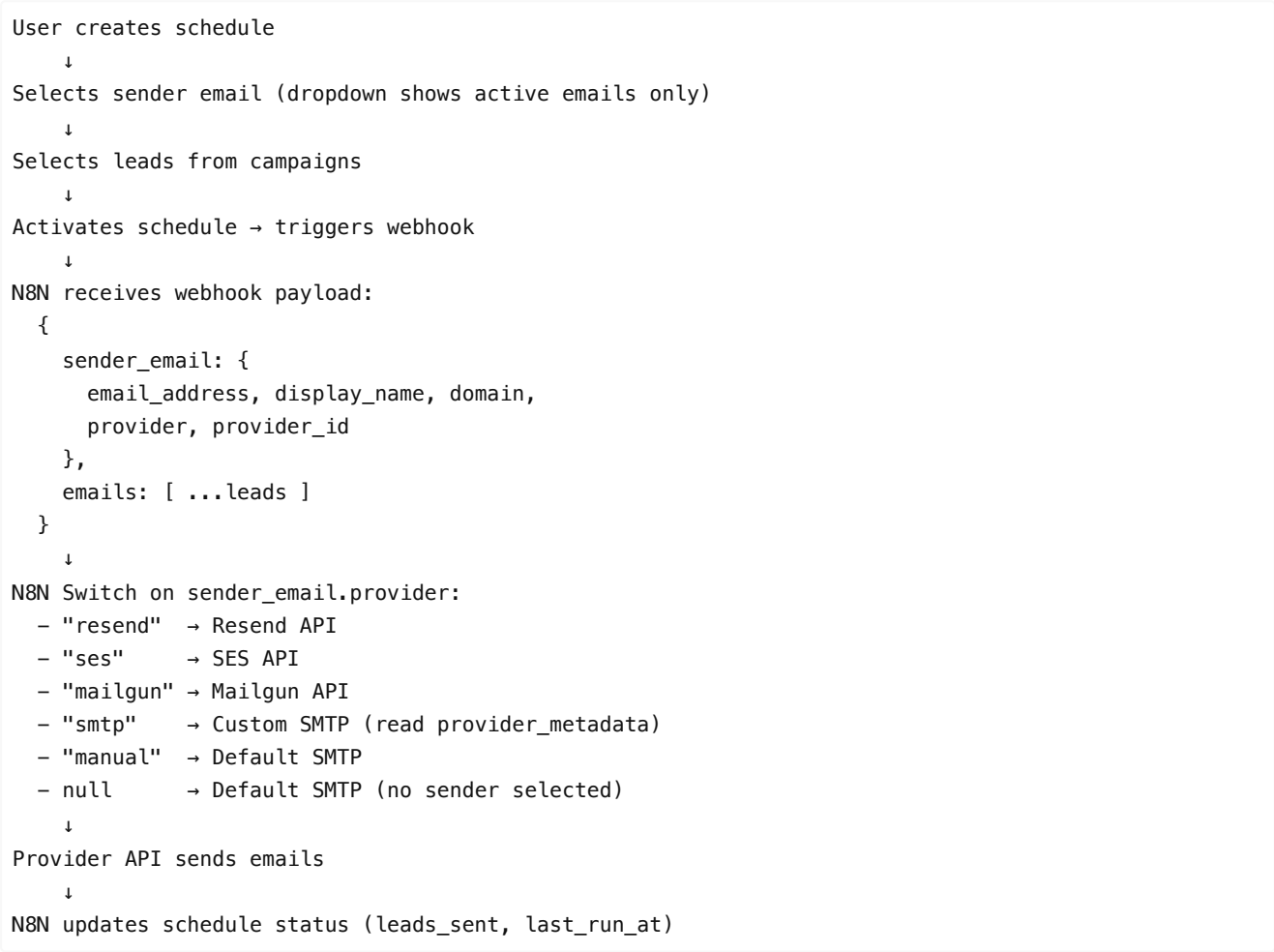
Example cron: every 15 minutes.

```
SELECT id, provider, provider_id, provider_metadata
FROM sender_emails
WHERE provider != 'manual'
AND (last_synced_at IS NULL OR last_synced_at < now() - interval '15 minutes');
```

5.6. Delete / Set Default

Same as before — see constraints section below.

6. Complete Dispatch Flow



7. Migration Checklist

- ☐ Run SQL migration `20260412120000_create_sender_emails_table.sql`
- ☐ Verify RLS policies are active on `sender_emails`
- ☐ Update N8N webhook processing to read `sender_email` object
- ☐ Implement Switch node for provider routing
- ☐ Configure API credentials for each provider (Resend key, SES credentials, etc.)
- ☐ Implement provider sync workflow (optional, for status verification)
- ☐ Test: create sender email with provider → trigger schedule → verify correct API is called
- ☐ Test: fallback when no sender email is selected
- ☐ Test: delete sender email → verify schedule's `sender_email_id` becomes null

8. Constraints & Edge Cases

Scenario	Behavior
----------	----------

User deletes a sender email used in a schedule	<code>sender_email_id</code> becomes <code>null</code> (ON DELETE SET NULL)
Duplicate email address per user	Rejected by unique constraint
Two defaults per user	Prevented by partial unique index
No sender email selected for schedule	<code>sender_email</code> in payload is <code>null</code> — use default SMTP
First email added by user	Automatically set as <code>is_default = true</code>
Sender email with <code>status != active</code>	Not shown in schedule/lead dropdowns — only active emails are selectable
Provider sync finds email suspended	N8N updates <code>status = 'suspended'</code> — UI reflects immediately
<code>provider_metadata</code> empty	Default <code>{}</code> — N8N falls back to env-level credentials